

Submission Worksheet

Submission Data

Course: IT202-010-S2026

Assignment: IT202 Milestone 2

Student: Joshua T. (jlt36)

Status: Submitted | **Worksheet Progress:** 100%

Potential Grade: 10.00/10.00 (100.00%)

Received Grade: 0.00/10.00 (0.00%)

Started: 4/22/2026 4:32:10 PM

Updated: 4/24/2026 2:09:41 AM

Grading Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-2/grading/jlt36>

View Link: <https://learn.ethereallab.app/assignment/v3/IT202-010-S2026/it202-milestone-2/view/jlt36>

Instructions

- Overview Link: https://youtu.be/t6X8dzZt_a0
- 1. Refer to Milestone2 of this doc: <https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
- 2. Ensure you read all instructions and objectives before starting.
- 3. Ensure you've gone through each lesson related to this Milestone
- 4. Switch to the Milestone2 branch
 - 1. `git checkout Milestone2` (ensure proper starting branch)
 - 2. `git pull origin Milestone2` (ensure history is up to date)
- 5. Fill out the below worksheet
 - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
 - Ensure proper styling is applied to each page
 - Ensure there are no visible technical errors; only user-friendly messages are allowed
- 6. Once finished, click "Submit and Export"
- 7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
 - 1. `git add .`
 - 2. `git commit -m "adding PDF"`
 - 3. `git push origin Milestone2`
 - 4. On Github merge the pull request from Milestone2 to qa
 - 5. On Github create a pull request from qa to prod and immediately merge. (This will trigger the prod deploy to make the Render.com prod links work)
- 8. Upload the same PDF to Canvas
- 9. Sync Local
 - 1. `git checkout qa`
 - 2. `git pull origin qa`

Section #1: (2 pts.) Data

Progress: 100%

≡ Task #1 (0.50 pts) - API Data Table

Part 1:

Progress: 100%

Details:

- Show the table(s) structure of the table made to hold your API
- There should be at least 3 properties/fields beyond `id`, `created`, `modified`, and respective `is_api` or `api_id` columns
- Logical types and constraints should be set
- Show at least a few records from each API table (samples should include both API fetched data and custom entries noted by an `api_id` or `is_api` column)

code snippet

id	title	url	author	created	modified	is_api
1	Malware concepts	https://www.malwarebytes.com	malware	2020-04-22 12:07:46	2020-04-22 12:07:46	1
4	How to protect your network	https://www.cisco.com/c/en/us/solutions/enterprise-networks/sec/how-to-protect-your-network.html	network	2020-04-22 12:07:46	2020-04-22 12:07:46	1
11	How to protect your network	https://www.cisco.com/c/en/us/solutions/enterprise-networks/sec/how-to-protect-your-network.html	network	2020-04-22 12:07:46	2020-04-22 12:07:46	0

API fetched data and custome entries

Saved: 4/22/2026 5:46:54 PM

Part 2:

Progress: 100%

Details:

- Add a direct link to your sql file from Github (should end in `.sql`)

URL #1

https://github.com/Joshua-Emman/jlt36-IT202-010/blob/Milestone2/public_html/



URL

<https://github.com/Joshua-Emma>

Progress: 100%

- Briefly note what each field/property represents

Each field in the Memes table stores a different part of the meme data. `id` is just the unique ID for each meme. `title` is the caption, `post_link` is the link to the original post (and it's unique so we don't get duplicates), and `image_url` is the actual image link. `author` is who posted it, and `subreddit` is where it came from. `is_nsfw` and `spoiler` are just flags for those types of content. `created` and `modified` keep track of when the record was made and last updated. `is_api` tells whether the meme came from the API (1) or was created manually (0).

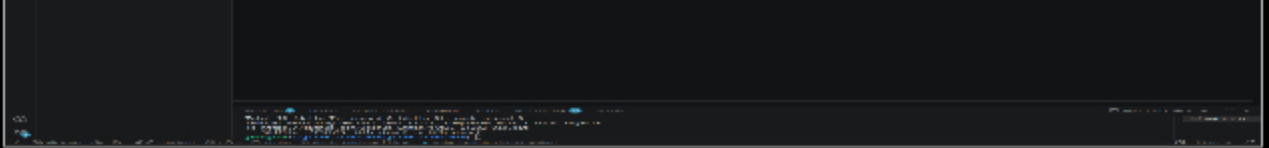
Progress: 100%

Progress: 100%

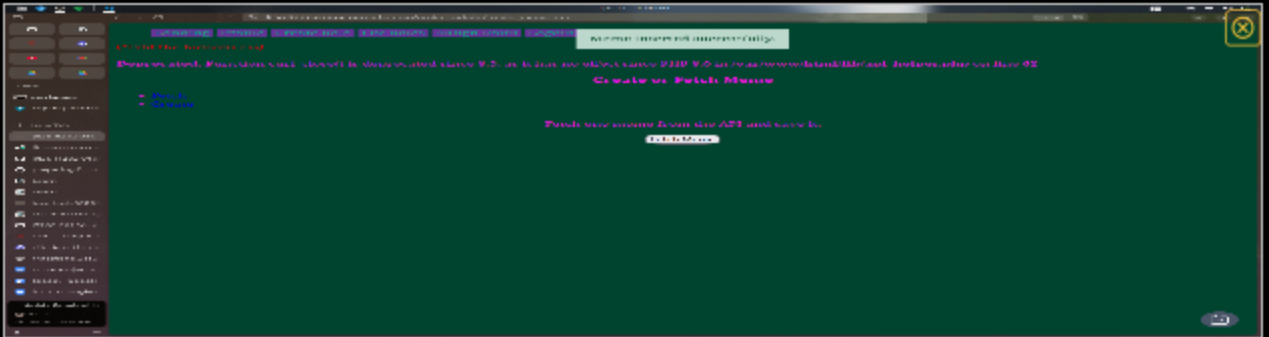
- Show the code that handles the data transformation/conversion (from the API)
- Show a Render.com qa example of API getting fetched and loaded into the DB

code snippet

[illegible]



db before fetch



successful fetch



db after fetch

 Saved: 4/22/2026 8:56:48 PM

Part 2:

Progress: 100%

Details:

- Include Render.com prod link to API fetch page
- Include pull request link for this feature

URL #1

[https://jlt36-it202-010-qa.onrender.com/
project/admin/create_meme.php](https://jlt36-it202-010-qa.onrender.com/project/admin/create_meme.php)



URL

<https://jlt36-it202-010-qa.onrende>



URL #2

[https://github.com/Joshua-Emman/jlt36-
IT202-010/pull/42](https://github.com/Joshua-Emman/jlt36-IT202-010/pull/42)



URL

<https://github.com/Joshua-Emma>



 Saved: 4/22/2026 8:56:48 PM

Part 3:

Progress: 100%

Details:

- Will you be using all the data or just a subset?
- Do you need to convert any data or extract pieces of a property's value?
- Briefly explain the transformation/extraction logic to get the API data to the shape your application intends to use

Your Response:

I'm only using part of the API data, not all of it. My app just needs the fields that match my Memes table, like the title, post link, image URL, author, subreddit, NSFW flag, and spoiler flag. Some of the data also has to be cleaned up a little before saving it. For example, the code checks a few possible field names for the post link and image URL, and if the post link is missing or incomplete, it creates a valid one so the meme can still be stored. It also changes values like NSFW and spoiler into 1 or 0, and adds `is_api = 1` so API memes can be told apart from manual ones. This makes the API data fit the format my app actually uses.



Saved: 4/22/2026 8:56:48 PM

⇒ Task #3 (0.50 pts.) - API Duplicates

Progress: 100%

Details:

- Consider how duplicate entries are handled
 - What happens if you get the same result from the API for an entity that exists but has been unmodified?
 - What happens if you get the same result from the API for an entity that exists but has been modified?

Your Response:

If the API returns a meme that already exists in the database and it has not been modified my app treats it as a duplicate and skips it instead of inserting another copy. This prevents repeated API results from filling the table with the same meme over and over. If the API returns a meme that already exists but the existing record was modified, my current logic still treats it as a duplicate based on the title and image URL, so it also gets skipped. Right now I'm not automatically updating existing records with API data. I chose that approach so manually changed data does not get overwritten by a repeated API fetch



Saved: 4/22/2026 8:58:19 PM

⇒ Task #4 (0.50 pts.) - API Fetching

Progress: 100%

Details:

- Consider how your application will trigger this data fetch
 - (i.e., periodic via cron job, poor man's cron job, on app start/awake, etc)
 - (i.e., User or Admin triggered via a specific page/action)
 - (i.e., Lazy loaded during searches for stale/missing content)

Your Response:

the API fetch is triggered by an Admin through a specific page and action. I made a create/fetch meme page where the Admin can press the fetch button to pull in a meme from the API and save it to the database. I chose this approach because it gives more control over when data is added and makes it easier to test and show for the milestone. Right now it is not set up as a cron job or automatic background task, and it is also not lazy loaded during searches.

 Saved: 4/22/2026 8:59:00 PM

Section #2: (1.5 pts.) Data Creation

Progress: 100%

≡ Task #1 (0.60 pts.) - Validations

Progress: 100%

Part 1:

Progress: 100%

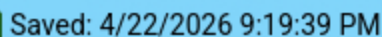
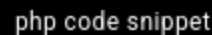
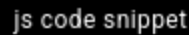
Details:

- Show the html validations (code)
- Show the javascript validations (code)
- Show the php validations (code)

```
1 <!-- HTML Validation -->
2 <!-- HTML Validation -->
3 <!-- HTML Validation -->
4 <!-- HTML Validation -->
5 <!-- HTML Validation -->
6 <!-- HTML Validation -->
7 <!-- HTML Validation -->
8 <!-- HTML Validation -->
9 <!-- HTML Validation -->
10 <!-- HTML Validation -->
11 <!-- HTML Validation -->
12 <!-- HTML Validation -->
13 <!-- HTML Validation -->
14 <!-- HTML Validation -->
15 <!-- HTML Validation -->
16 <!-- HTML Validation -->
17 <!-- HTML Validation -->
18 <!-- HTML Validation -->
19 <!-- HTML Validation -->
20 <!-- HTML Validation -->
21 <!-- HTML Validation -->
22 <!-- HTML Validation -->
23 <!-- HTML Validation -->
24 <!-- HTML Validation -->
25 <!-- HTML Validation -->
26 <!-- HTML Validation -->
27 <!-- HTML Validation -->
28 <!-- HTML Validation -->
29 <!-- HTML Validation -->
30 <!-- HTML Validation -->
31 <!-- HTML Validation -->
32 <!-- HTML Validation -->
33 <!-- HTML Validation -->
34 <!-- HTML Validation -->
35 <!-- HTML Validation -->
36 <!-- HTML Validation -->
37 <!-- HTML Validation -->
38 <!-- HTML Validation -->
39 <!-- HTML Validation -->
40 <!-- HTML Validation -->
41 <!-- HTML Validation -->
42 <!-- HTML Validation -->
43 <!-- HTML Validation -->
44 <!-- HTML Validation -->
45 <!-- HTML Validation -->
46 <!-- HTML Validation -->
47 <!-- HTML Validation -->
48 <!-- HTML Validation -->
49 <!-- HTML Validation -->
50 <!-- HTML Validation -->
51 <!-- HTML Validation -->
52 <!-- HTML Validation -->
53 <!-- HTML Validation -->
54 <!-- HTML Validation -->
55 <!-- HTML Validation -->
56 <!-- HTML Validation -->
57 <!-- HTML Validation -->
58 <!-- HTML Validation -->
59 <!-- HTML Validation -->
60 <!-- HTML Validation -->
61 <!-- HTML Validation -->
62 <!-- HTML Validation -->
63 <!-- HTML Validation -->
64 <!-- HTML Validation -->
65 <!-- HTML Validation -->
66 <!-- HTML Validation -->
67 <!-- HTML Validation -->
68 <!-- HTML Validation -->
69 <!-- HTML Validation -->
70 <!-- HTML Validation -->
71 <!-- HTML Validation -->
72 <!-- HTML Validation -->
73 <!-- HTML Validation -->
74 <!-- HTML Validation -->
75 <!-- HTML Validation -->
76 <!-- HTML Validation -->
77 <!-- HTML Validation -->
78 <!-- HTML Validation -->
79 <!-- HTML Validation -->
80 <!-- HTML Validation -->
81 <!-- HTML Validation -->
82 <!-- HTML Validation -->
83 <!-- HTML Validation -->
84 <!-- HTML Validation -->
85 <!-- HTML Validation -->
86 <!-- HTML Validation -->
87 <!-- HTML Validation -->
88 <!-- HTML Validation -->
89 <!-- HTML Validation -->
90 <!-- HTML Validation -->
91 <!-- HTML Validation -->
92 <!-- HTML Validation -->
93 <!-- HTML Validation -->
94 <!-- HTML Validation -->
95 <!-- HTML Validation -->
96 <!-- HTML Validation -->
97 <!-- HTML Validation -->
98 <!-- HTML Validation -->
99 <!-- HTML Validation -->
100 <!-- HTML Validation -->
```

html code snippet

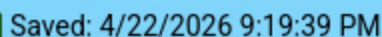
```
1 <!-- HTML Validation -->
2 <!-- HTML Validation -->
3 <!-- HTML Validation -->
4 <!-- HTML Validation -->
5 <!-- HTML Validation -->
6 <!-- HTML Validation -->
7 <!-- HTML Validation -->
8 <!-- HTML Validation -->
9 <!-- HTML Validation -->
10 <!-- HTML Validation -->
11 <!-- HTML Validation -->
12 <!-- HTML Validation -->
13 <!-- HTML Validation -->
14 <!-- HTML Validation -->
15 <!-- HTML Validation -->
16 <!-- HTML Validation -->
17 <!-- HTML Validation -->
18 <!-- HTML Validation -->
19 <!-- HTML Validation -->
20 <!-- HTML Validation -->
21 <!-- HTML Validation -->
22 <!-- HTML Validation -->
23 <!-- HTML Validation -->
24 <!-- HTML Validation -->
25 <!-- HTML Validation -->
26 <!-- HTML Validation -->
27 <!-- HTML Validation -->
28 <!-- HTML Validation -->
29 <!-- HTML Validation -->
30 <!-- HTML Validation -->
31 <!-- HTML Validation -->
32 <!-- HTML Validation -->
33 <!-- HTML Validation -->
34 <!-- HTML Validation -->
35 <!-- HTML Validation -->
36 <!-- HTML Validation -->
37 <!-- HTML Validation -->
38 <!-- HTML Validation -->
39 <!-- HTML Validation -->
40 <!-- HTML Validation -->
41 <!-- HTML Validation -->
42 <!-- HTML Validation -->
43 <!-- HTML Validation -->
44 <!-- HTML Validation -->
45 <!-- HTML Validation -->
46 <!-- HTML Validation -->
47 <!-- HTML Validation -->
48 <!-- HTML Validation -->
49 <!-- HTML Validation -->
50 <!-- HTML Validation -->
51 <!-- HTML Validation -->
52 <!-- HTML Validation -->
53 <!-- HTML Validation -->
54 <!-- HTML Validation -->
55 <!-- HTML Validation -->
56 <!-- HTML Validation -->
57 <!-- HTML Validation -->
58 <!-- HTML Validation -->
59 <!-- HTML Validation -->
60 <!-- HTML Validation -->
61 <!-- HTML Validation -->
62 <!-- HTML Validation -->
63 <!-- HTML Validation -->
64 <!-- HTML Validation -->
65 <!-- HTML Validation -->
66 <!-- HTML Validation -->
67 <!-- HTML Validation -->
68 <!-- HTML Validation -->
69 <!-- HTML Validation -->
70 <!-- HTML Validation -->
71 <!-- HTML Validation -->
72 <!-- HTML Validation -->
73 <!-- HTML Validation -->
74 <!-- HTML Validation -->
75 <!-- HTML Validation -->
76 <!-- HTML Validation -->
77 <!-- HTML Validation -->
78 <!-- HTML Validation -->
79 <!-- HTML Validation -->
80 <!-- HTML Validation -->
81 <!-- HTML Validation -->
82 <!-- HTML Validation -->
83 <!-- HTML Validation -->
84 <!-- HTML Validation -->
85 <!-- HTML Validation -->
86 <!-- HTML Validation -->
87 <!-- HTML Validation -->
88 <!-- HTML Validation -->
89 <!-- HTML Validation -->
90 <!-- HTML Validation -->
91 <!-- HTML Validation -->
92 <!-- HTML Validation -->
93 <!-- HTML Validation -->
94 <!-- HTML Validation -->
95 <!-- HTML Validation -->
96 <!-- HTML Validation -->
97 <!-- HTML Validation -->
98 <!-- HTML Validation -->
99 <!-- HTML Validation -->
100 <!-- HTML Validation -->
```



Progress: 100%

- Briefly explain the html validations used
- Briefly explain the javascript validation logic
- Briefly explain the php validation logic
- Note which role(s) have access to this create form

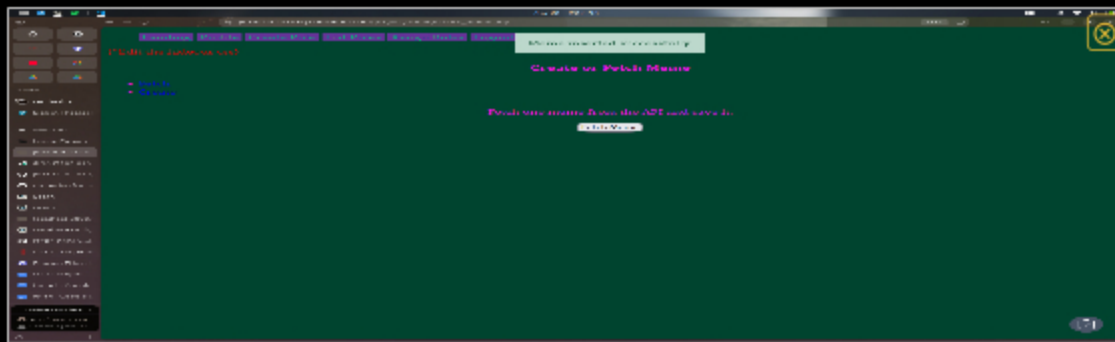
The HTML validation uses built in form rules like required, type="url", and maxlength to make sure the user fills in the important fields and enters data in the right format before submitting. The JavaScript validation runs when the form is submitted and checks things like whether required fields are empty and whether text fields go over the allowed length. The PHP validation does the same checks again on the server side, including required fields, URL format, and length limits, so invalid data cannot be saved even if client side validation is bypassed. This create form is only available to users with the Admin role.



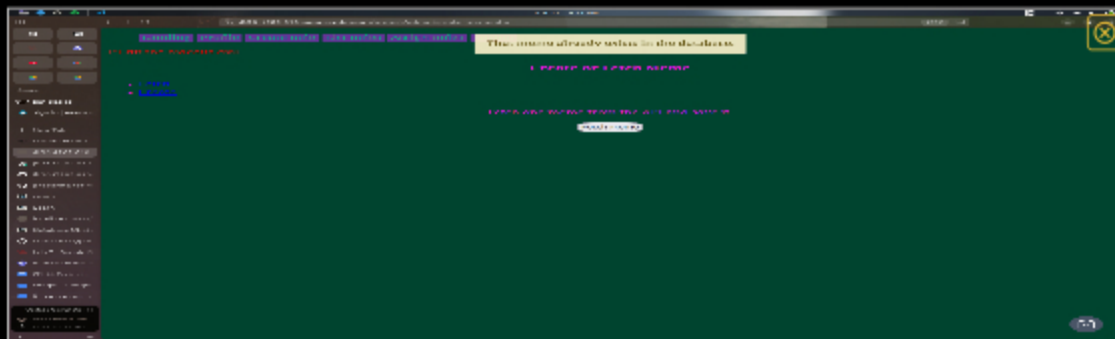
Progress: 100%

Details:

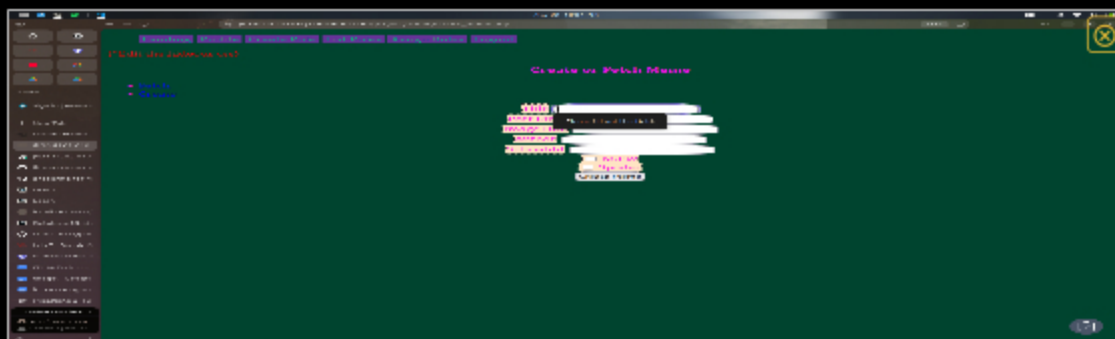
- Show a successful creation
- Show how a duplicate submission is handled (i.e., what message or behavior occurs)
- Demonstrate various validation errors
- Ensure Render.com prod url is visible



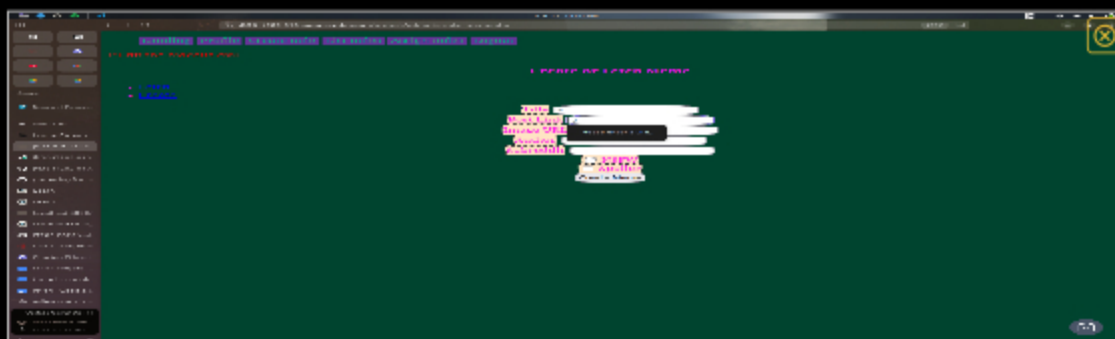
successful meme creation



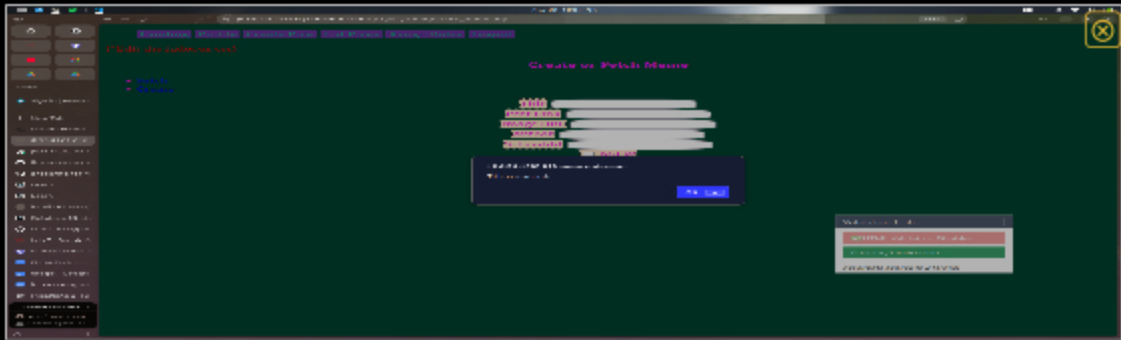
dupe submission being handled



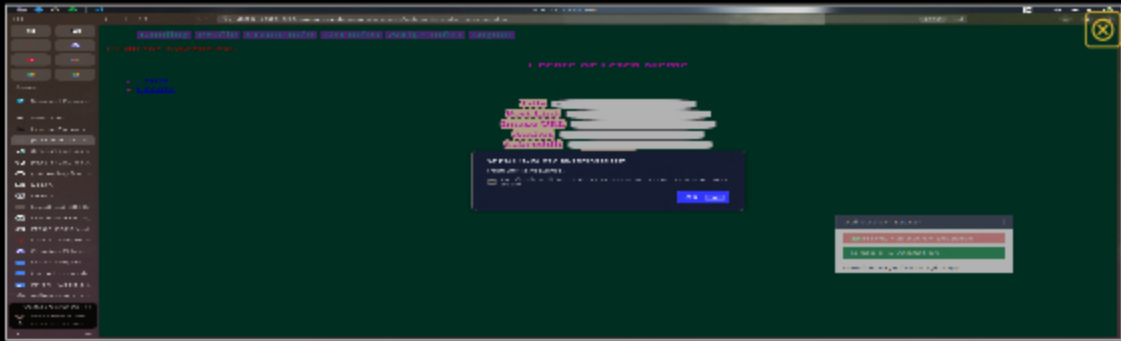
html empty title



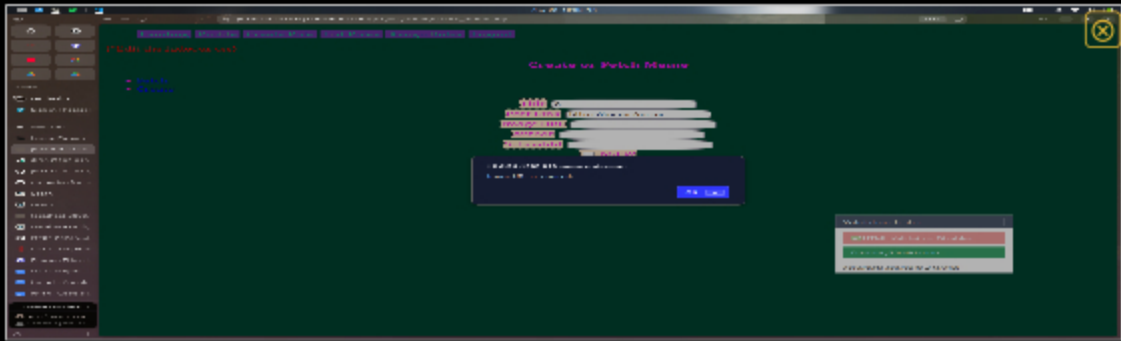
html not a url



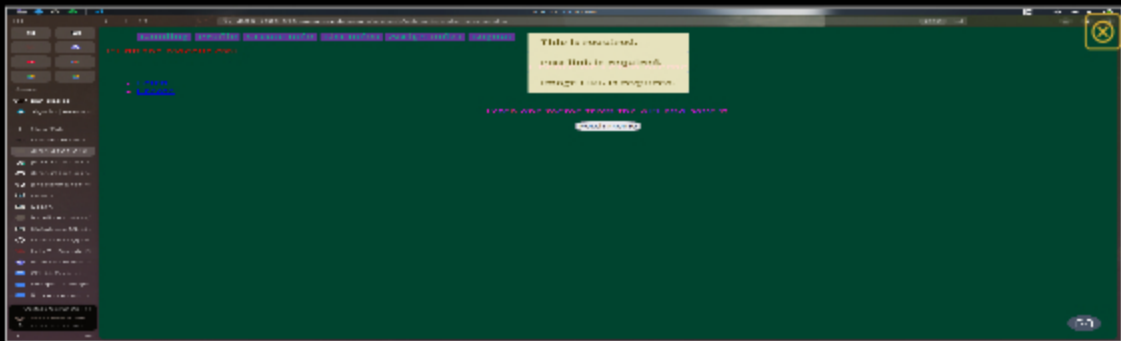
js empty title



js invalid post link



js empty image url



php validation errors



Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

https://jlt36-it202-010-qa.onrender.com/project/admin/create_meme.php



URL

<https://jlt36-it202-010-qa.onrender.com/>



URL #2

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/42/changes/>



URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/42/changes/>

f752b751a13b5dce57d1b312e0f4ef96ed3ab9b6#diff-5b360a0c52e0f75e5aaffc131ada9ccd6377afb8beca



Saved: 4/22/2026 10:16:59 PM

Section #3: (1.5 pts.) Data List Page (Many Items)

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles the filter/sort logic
- Show the code that enforces the record limit (valid range 1-100, default 10 if out of range)
- Show the code that generates the list/grid output
- Styling must be applied

```
// UCID: jlt36
// Date: 04/22/2026
```

filter and sort logic

```
19
20 // UCID: jlt36
21 // Date: 04/22/2026
```


limit logic

list/grid output

Progress: 100%

- The filter and sort logic just takes whatever the user picks in the form, like the title, subreddit, source, sort field, and sort order, and uses that to build the query. If a filter is filled in, it adds that condition to the WHERE part of the SQL query, and then it adds the selected ORDER BY option to sort the results.
- The limit is checked on the server side by turning it into a number and making sure it stays between 1 and 100. If the user enters something outside that range, it just falls back to 10. Then that value gets used in the query with LIMIT.
- The list output is made by looping through the results and creating one card for each meme. Each card shows a short summary, like the title, author, subreddit, NSFW and spoiler flags, and whether it came from the API or was made manually. It also includes the view, edit, and delete links for that meme.
- For styling, I used bootstrap classes to make the page look more organized and easier to use. I used a grid layout with cards, spacing between items, styled form fields, and colored buttons for the actions



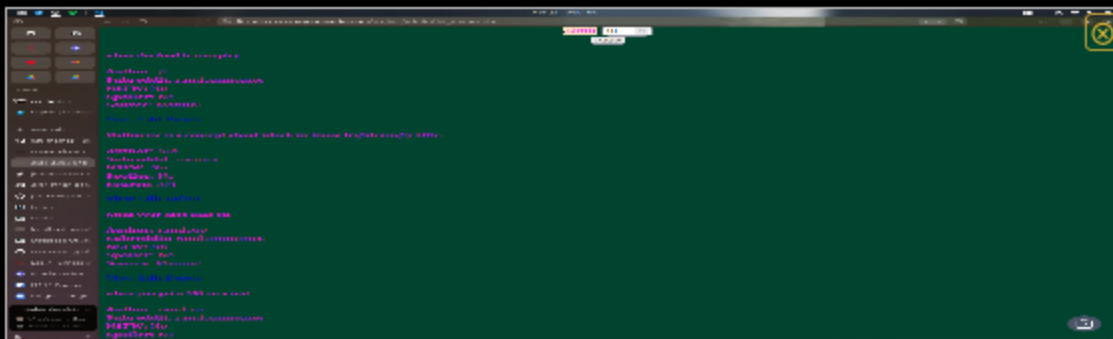
Saved: 4/23/2026 8:57:26 PM

Task #2 (0.60 pts.) - Examples

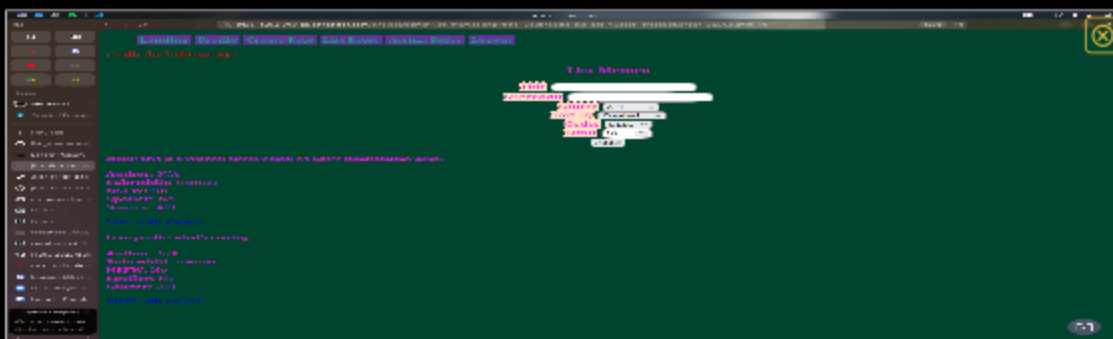
Progress: 100%

Details:

- Show a few variations of filter/sort applied (including no records - should display a "no results" message)
- Show both API-fetched and manually created entities present in the list
- Show the limit field in use with at least two different values
- Each list item should be a summary
- Each list item should have the following links
 - A link to a single view of the specific entity (i.e., a details page)
 - A link to delete this entity (this may be an admin-only functionality, but it should be present for the respective role)
 - A link to edit this entity (this may be an admin-only functionality, but it should be present for the respective role)
- Ensure Render.com prod url is visible



normal list view

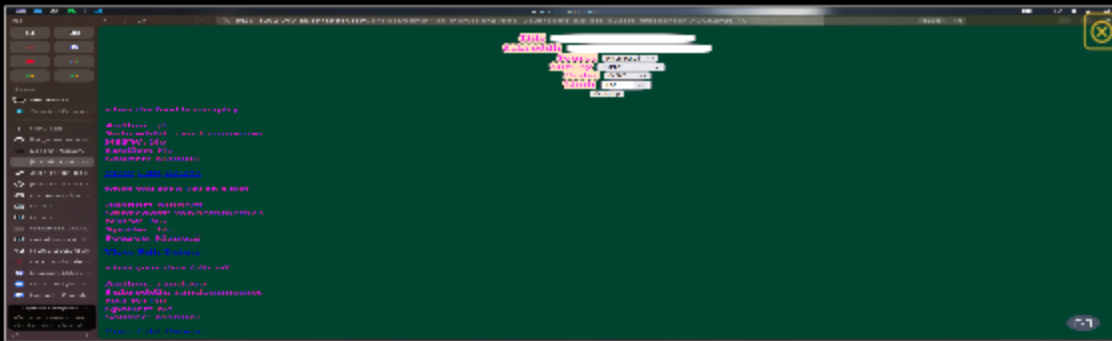


api only filter

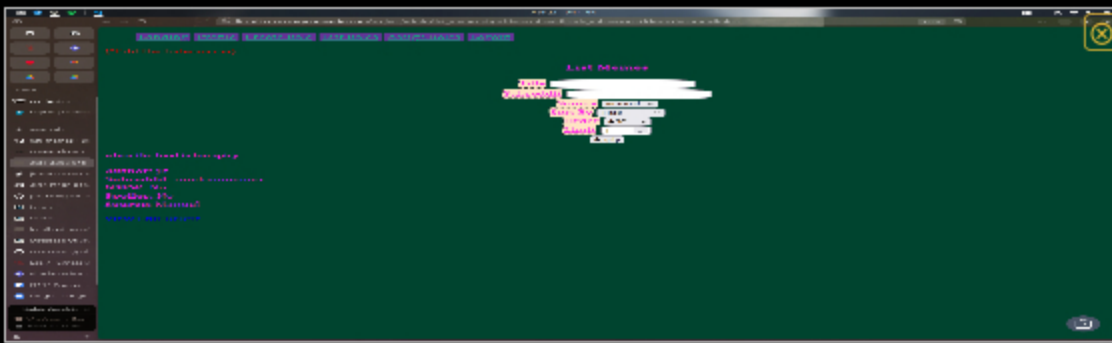




no results



some variation in filter



limiting to only 1 result



Saved: 4/23/2026 9:15:08 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

<https://github.com/Joshua-Emman/jlt36-it202-010/pull/44>



URL

<https://github.com/Joshua-Ei>



URL #2

https://jlt36-it202-010-qa.onrender.com/project/admin/list_memes.php



URL

<https://jlt36-it202-010-qa.onr>



Saved: 4/23/2026 9:16:16 PM

Section #4: (1.5 pts.) View Details Page (Single Item)

Progress: 100%

≡ Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect back to the previous page with an appropriate message)
- Show the code that generates the entity output
- Styling must be applied

```
// UCID: jlt36
// Date: 04/22/2026
// Description: Retrieves the meme id from the query string and redirects back if it is missing or invalid.
$id = (int)($_GET["id"] ?? 0);

if ($id <= 0) {
    flash("Invalid or missing meme id.", "warning");
    $back = $_SERVER["HTTP_REFERER"] ?? get_url("admin/list_memes.php");
    die(header("Location: " . $back));
} <- #14-18 if ($id <= 0)
```

id retrieval + invalid missing id handling

loading of record snippet



⇒ Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the entity output is generated
- Briefly note the styling choices

Your Response:

- The loading logic starts by getting the meme id from the URL using `$_GET`. It converts that value into an integer and checks if it is valid. If the id is missing or invalid, the page redirects back to the previous page and shows a user friendly message. If the id is valid, the code runs a query to load that one meme from the database. If no matching record is found, it also redirects back with a message instead of showing an error.
- The entity output is generated by taking the single record from the query result and displaying its fields on the page. It shows more details than the list page, like the title, author, subreddit, post link, image link, NSFW flag, spoiler flag, source, and timestamps. It also includes action links for editing and deleting the meme, and a link to go back to the list page.
- For styling, I used bootstrap classes again to make the details page look cleaner and easier to read. I used a container for layout, a card for the main content, spacing and shadow classes for better structure, and colored buttons for the actions.

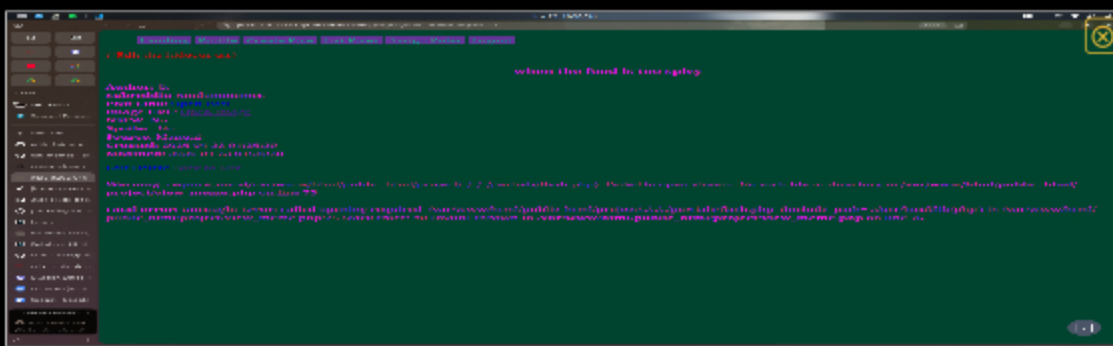


Task #2 (0.60 pts.) - Examples

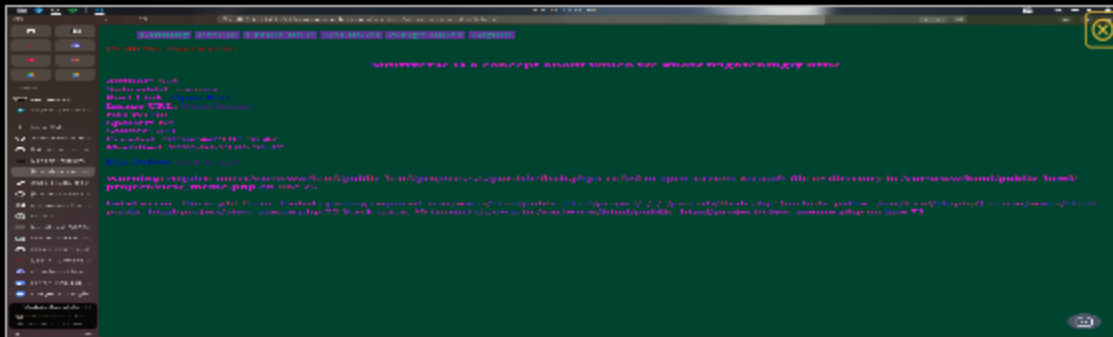
Progress: 100%

Details:

- Show a few examples of different entities
- Each entity item should have more details than the summary view (if possible)
- Each entity should have the following links
 - A link to edit the single entity (this may be an admin-only thing, but it should be present for the respective role)
 - A link to delete the single entity (this may be an admin-only thing, but it should be present for the respective role)



viewing a manual meme



viewing an api meme



Saved: 4/23/2026 10:31:29 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

<https://github.com/Joshua-Emman/jlt36-it202-010/pull/45>



URL

<https://github.com/Joshua-Ei>



URL #2

https://jlt36-it202-010-qa.onrender.com/project/view_meme.php?id=17



URL

<https://jlt36-it202-010-qa.onrender.com/>



URL #3

https://jlt36-it202-010-qa.onrender.com/project/view_meme.php?id=16



URL

<https://jlt36-it202-010-qa.onrender.com/>



Saved: 4/23/2026 10:32:54 PM

Section #5: (1.5 pts.) Edit Page

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles loading of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids
- Show the code that generates the form with the correct data types and populates it
- Show the html, javascript, and php validation (should be similar to create)
- Styling must be applied

```
// DATE: 11/18
// Date: 04/23/2020
// DESCRIPTION: updates the record id from the query string and uses it to load or update the selected record
$id = $_GET['id']; if ($id == '') $id = 1;
if ($id == 0) {
    $error = "Invalid or missing record id.", "warning";
    $url = "index.php?edit=1";
}
$record = [];
$id = $id;
$query = "SELECT title, post, link, image url, author, authorid, is active, expires FROM `T002-025-Photos` WHERE id = '$id' LIMIT 1";
try {
    $stmt = $db->prepare($query);
    $stmt->execute();
    $row = $stmt->fetch();
    if ($row) {
        $record = $row;
    } else {
        $error = "Record not found.", "warning";
        $url = "index.php?edit=1";
    }
} catch (PDOException $e) {
    $error = "Error fetching record.", "warning";
    $url = "index.php?edit=1";
}
if ($error) {
    $url = "index.php?edit=1";
}
```

loading the record, id retrieval, invalid/missing id handling

```
// DATE: 11/18
// Date: 04/23/2020
// DESCRIPTION: client side validation for the edit record form before submission
function validateForm() {
    var title = form.title.value.trim();
    var postLink = form.postLink.value.trim();
    var imageurl = form.imageurl.value.trim();
    var author = form.author.value.trim();
    var authorid = form.authorid.value.trim();
    var isactive = form.isactive.value.trim();
    var expires = form.expires.value.trim();
    if (title.length < 1) {
        alert("Title is required.");
        return false;
    }
    if (postLink.length < 1) {
        alert("Post Link is required.");
        return false;
    }
    if (imageurl.length < 1) {
        alert("Image URL is required.");
        return false;
    }
    if (author.length < 1) {
        alert("Author must be 100 characters or fewer.");
        return false;
    }
    if (authorid.length < 1) {
        alert("Authorid must be 100 characters or fewer.");
        return false;
    }
    if (isactive.length < 1) {
        alert("Is active must be 1 or 0.");
        return false;
    }
    if (expires.length < 1) {
        alert("Expires must be 100 characters or fewer.");
        return false;
    }
    return true;
}
```

form generation, html validation, styling

```
// DATE: 11/18
// Date: 04/23/2020
// DESCRIPTION: client side validation for the edit record form before submission
function validateForm() {
    var title = form.title.value.trim();
    var postLink = form.postLink.value.trim();
    var imageurl = form.imageurl.value.trim();
    var author = form.author.value.trim();
    var authorid = form.authorid.value.trim();
    var isactive = form.isactive.value.trim();
    var expires = form.expires.value.trim();
    if (title.length < 1) {
        alert("Title is required.");
        return false;
    }
    if (postLink.length < 1) {
        alert("Post Link is required.");
        return false;
    }
    if (imageurl.length < 1) {
        alert("Image URL is required.");
        return false;
    }
    if (author.length < 1) {
        alert("Author must be 100 characters or fewer.");
        return false;
    }
    if (authorid.length < 1) {
        alert("Authorid must be 100 characters or fewer.");
        return false;
    }
    if (isactive.length < 1) {
        alert("Is active must be 1 or 0.");
        return false;
    }
    if (expires.length < 1) {
        alert("Expires must be 100 characters or fewer.");
        return false;
    }
    return true;
}
```

js validation

```
// DATE: 11/18
// Date: 04/23/2020
// DESCRIPTION: client side validation for the edit record form before updating the database
function validateForm() {
    var title = form.title.value.trim();
    if (title.length < 1) {
        alert("Title is required.");
        return false;
    }
    return true;
}
```



```

if (strlen($name["title"]) > 544) {
    $errors[] = "Title must be 544 characters or fewer.";
}

if (empty($name["post_link"])) {
    $errors[] = "Post link is required.";
} else {
    if (!filter_var($name["post_link"], FILTER_VALIDATE_URL)) {
        $errors[] = "Post link must be a valid URL.";
    }
}

if (empty($name["image_url"])) {
    $errors[] = "Image URL is required.";
} else {
    if (!filter_var($name["image_url"], FILTER_VALIDATE_URL)) {
        $errors[] = "Image URL must be a valid URL.";
    }
}

if (empty($name["author"])) {
    $errors[] = "Author must be 100 characters or fewer.";
}

if (empty($name["subreddit"])) {
    $errors[] = "Subreddit must be 100 characters or fewer.";
}

```

php validation



Saved: 4/23/2026 11:23:30 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the loading logic works
- Briefly explain how the form is generated and submission is handled
- Briefly note the styling choices
- Note what fields are editable and why the choice was made for them (it's likely not all fields are editable, especially `id`, `created`, `modified`, etc)

Your Response:

- The loading logic starts by getting the meme id from the URL, and if the form is submitted it can also get the id from POST. The id gets turned into an integer and checked first so the page does not try to load a bad record. If the id is missing, invalid, or the meme is not found, the user gets sent back to the list page with a friendly message. If the id is valid, that meme's data is loaded from the database and used to fill in the edit form.
- The form is made by taking the current meme data and putting it into the input fields so the user can see what is already there before making changes. When the form is submitted, the JavaScript validation runs first, then the PHP validation runs on the server side, and if everything looks good the record gets updated in the database. After that, the page shows a success message and keeps the updated values in the form.
- For styling, I used bootstrap again so the page looks cleaner and matches the rest of the project. The form uses styled inputs, spacing, and a button so it is easier to read/use.
- The fields I made editable are title, post_link, image_url, author, subreddit, is_nsfw, and spoiler because those are the parts of the meme that someone might actually want to change. I did not make fields like id, created, or modified editable because those are handled by the system and should not be changed manually.



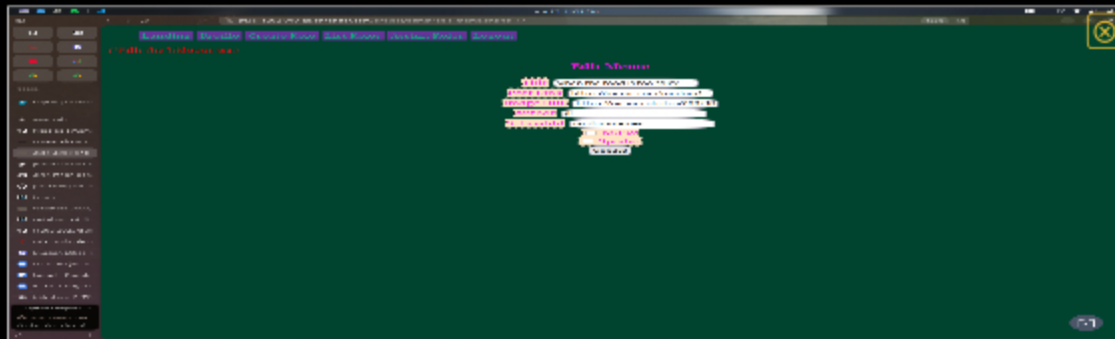
Saved: 4/23/2026 11:23:30 PM

Task #2 (0.60 pts.) - Examples

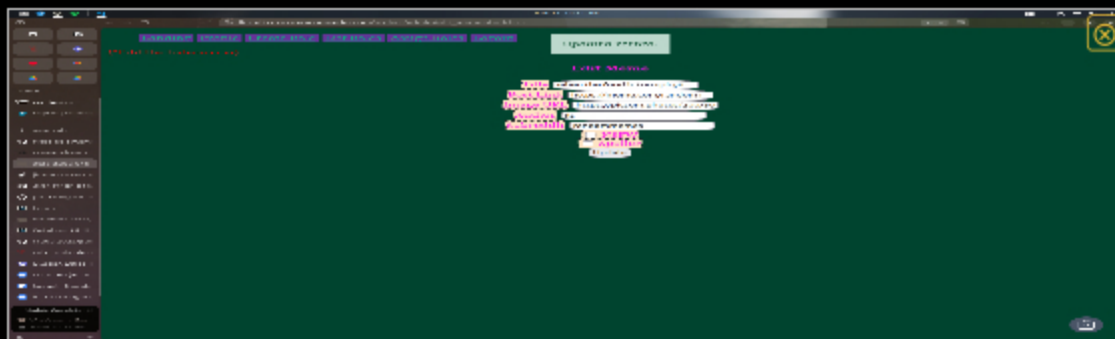
Progress: 100%

Details:

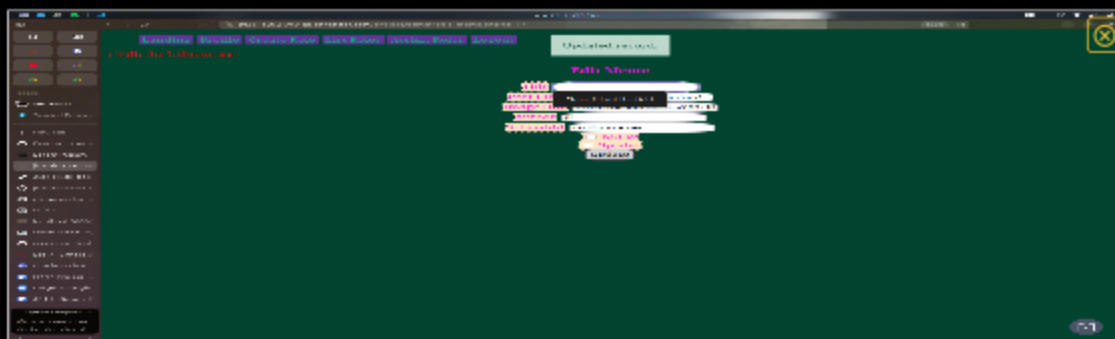
- Show a before and after of updating an entity (clearly caption)
- Successful update should have an appropriate message shown
- The updated data should be shown in the form
- Any errors should have user-friendly messages shown
- Show some examples of validation errors (html, javascript, php)



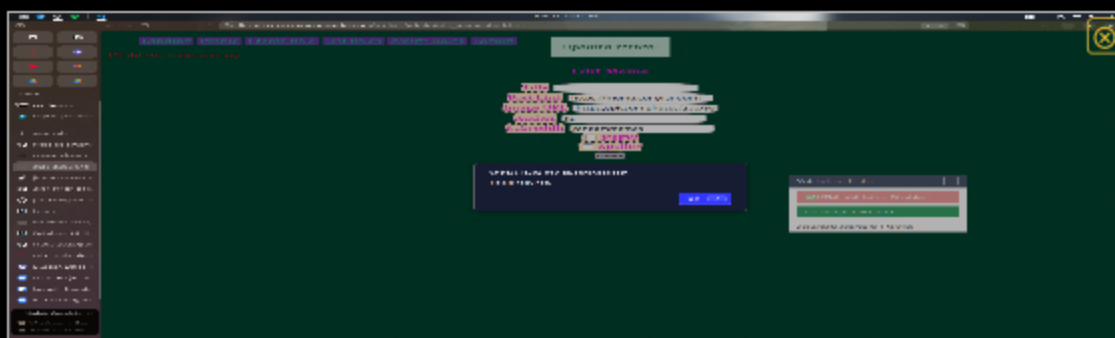
normal page after clicking edit



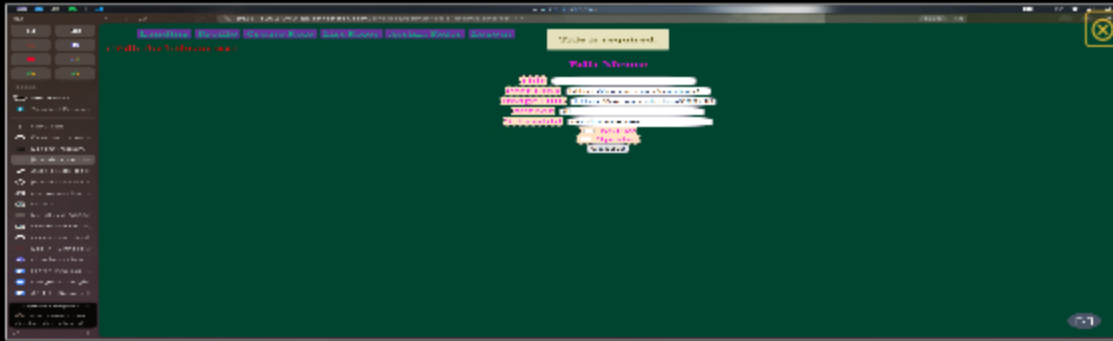
update successful



html empty title



js empty title



php empty title

 Saved: 4/23/2026 11:35:27 PM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page
- Include pull request link for this feature

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/46>



URL

<https://github.com/Joshua-Ei>



URL #2

https://jlt36-it202-010-qa.onrender.com/project/admin/edit_meme.php?id=17



URL

<https://jlt36-it202-010-qa.onr>



 Saved: 4/23/2026 11:36:25 PM

Section #6: (1.5 pts.) Delete Logic

Progress: 100%

Task #1 (0.60 pts.) - Code

Progress: 100%

Part 1:

Progress: 100%

Details:

- Show the code that handles deletion of the record (and how the id retrieved for usage)
 - Include the code that handles invalid/missing ids (should redirect to the list page with an appropriate message)

with an appropriate message)

- Handle any necessary roles/permissions (i.e., it's not likely that anyone can delete any entity they choose)
- Show the redirect logic that goes back to the previous page (where the route came from), if it was a list page, it should maintain the filter/sort criteria

```
<?php
require(__DIR__ . '/../../partials/nav.php');

if (!has_role("Admin")) {
    flash("You don't have permission to do that.", "warning");
    die(header("Location: " . get_url("landing.php")));
}

// UCID: jlt36
// Date: 04/23/2026
// Description: retrieves the meme id for deletion and redirects to the list page if the id is missing or invalid
$id = (int)($_GET["id"] ?? 0);

if ($id <= 0) {
    flash("Invalid or missing meme id.", "warning");
    die(header("Location: " . get_url("admin/list_memes.php")));
}
```

permission + id retrieval, invalid/missing id handling

```
// UCID: jlt36
// Date: 04/23/2026
// Description: Deletes the selected meme record and redirects back to the previous page.
$db = getDB();
$query = "DELETE FROM `IT202-M25-Memes` WHERE id = :id LIMIT 1";

try {
    $stmt = $db->prepare($query);
    $stmt->execute(["id" => $id]);

    if ($stmt->rowCount() > 0) {
        flash("Meme deleted successfully.", "success");
    } else {
        flash("Record not found.", "warning");
    }
} catch (PDOException $e) {
    error_log("Error deleting meme: " . var_export($e, true));
    flash("An error occurred while deleting the meme.", "danger");
}

$back = $_SERVER["HTTP_REFERER"] ?? get_url("admin/list_memes.php");
die(header("Location: " . $back));
```

delete and redirect logic



Saved: 4/23/2026 11:52:55 PM

Part 2:

Progress: 100%

Details:

- Briefly explain how the deletion logic works
- Is it a soft or hard delete?
- Note any permissions/rules applied.
- Briefly explain how the redirect maintains the previous route and any filter/sort data

Your Response:

- The delete logic works by getting the meme id from the URL, making sure it's valid, and then running a DELETE query for that record. If the id is missing or bad, the user gets sent back to the list page with a friendly message instead of the page trying to delete the wrong thing. If the delete works, it shows a success message, and if something goes wrong, it shows a user-friendly error.
- This is a hard delete because the record is actually removed from the database instead of just being marked as deleted.

just being marked as deleted.

- The main rule is that only Admins can delete memes. If someone without the Admin role tries to use that page, they get redirected away and shown a warning.
- The redirect uses the previous page URL with HTTP_REFERER, so if the user clicked delete from a filtered or sorted list page, it sends them back to that same page. That way the filter and sort stuff in the URL stays there



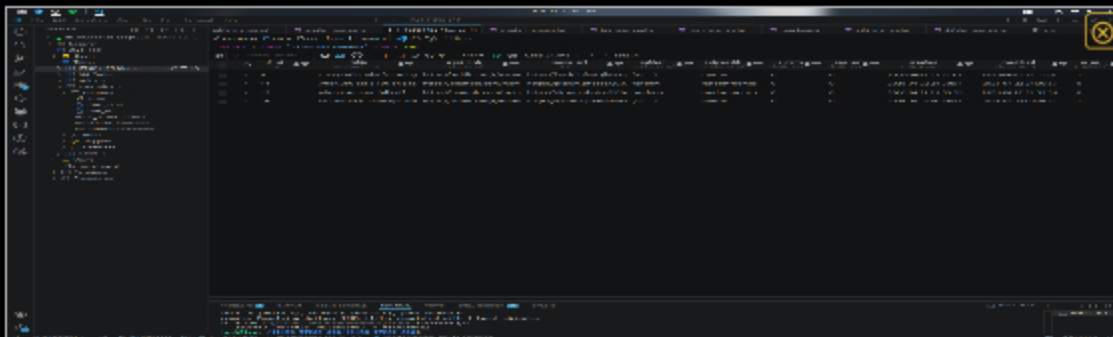
Saved: 4/23/2026 11:52:55 PM

Task #2 (0.60 pts.) - Examples

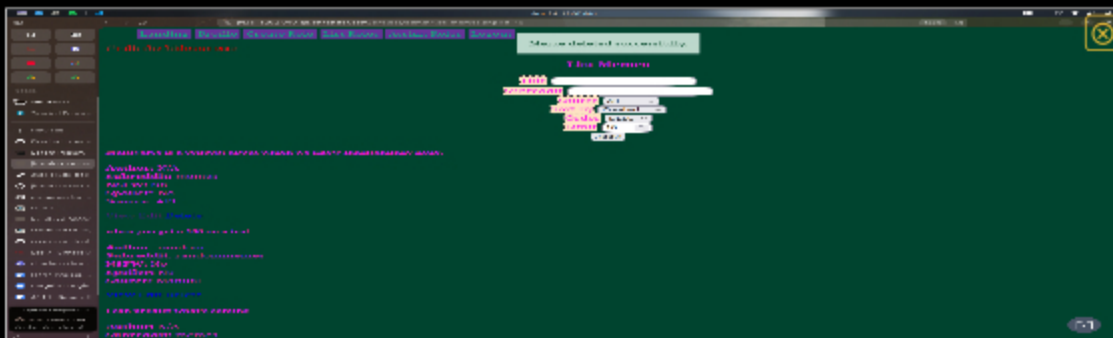
Progress: 100%

Details:

- Show a successful delete message
- Show a before and after database view of the record being deleted (clearly caption)
- Any errors should have user-friendly messages shown



before deletion



success message





db after deletion



Saved: 4/24/2026 12:03:15 AM

Task #3 (0.30 pts.) - Links

Progress: 100%

Details:

- Include the Render.com prod link to this page (even though it's not a full page, it's more like how logout works)
- Include pull request link for this feature

URL #1

https://jlt36-it202-010-qa.onrender.com/project/admin/delete_meme.php?id=15



URL

https://jlt36-it202-010-qa.onrender.com/project/admin/delete_meme.php?id=15



URL #2

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/47>



URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/47>



Saved: 4/24/2026 1:31:50 AM

Section #7: (0.5 pts.) Misc

Progress: 100%

Task #1 (0.17 pts.) - Github Details

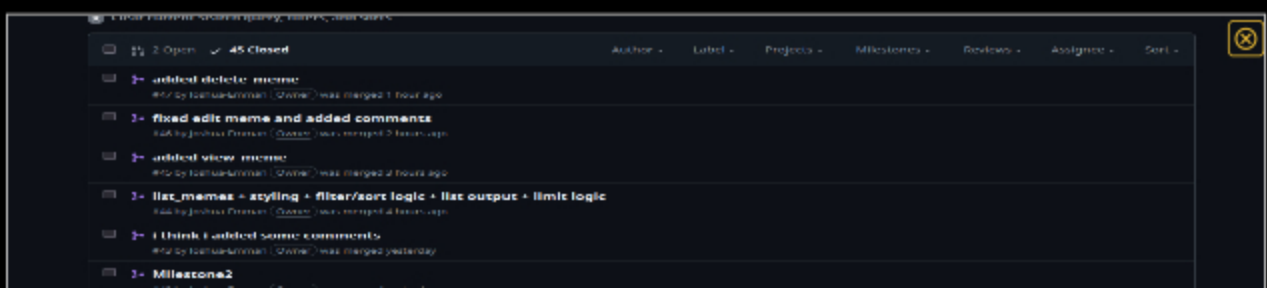
Progress: 100%

Part 1:

Progress: 100%

Details:

From the Commits tab of the Pull Request screenshot the commit history (it may not be possible to capture all, just grab a reasonable chunk)



→ preparing for milestone2, memo app, created table, admin files
sent by JoshuaEmman (Owner) was merged yesterday

commits history

📄 Saved: 4/24/2026 1:37:06 AM

Part 2:

Progress: 100%

Details:

Include the link to the Pull Request (should end in `/pull/#`) (Milestone2 into qa)

URL #1

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/47>



URL

<https://github.com/Joshua-Emman/jlt36-IT202-010/pull/47>

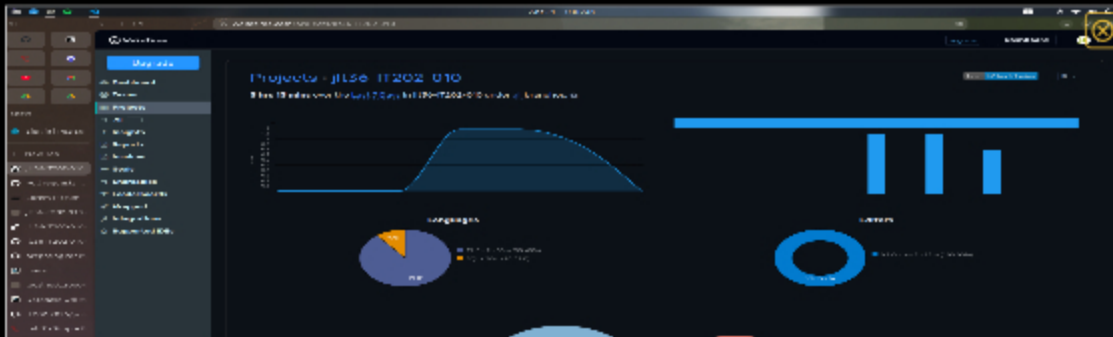
📄 Saved: 4/24/2026 1:37:06 AM

Task #2 (0.17 pts.) - WakaTime - Activity

Progress: 100%

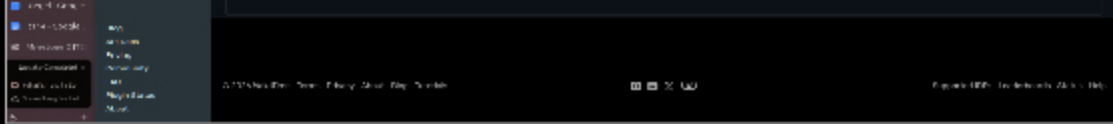
Details:

- Visit the WakaTime.com Dashboard
- Click **Projects** and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



overall time

File	Minutes
21:00 - 21:05	21:00 - 21:05
21:05 - 21:10	21:05 - 21:10
21:10 - 21:15	21:10 - 21:15
21:15 - 21:20	21:15 - 21:20
21:20 - 21:25	21:20 - 21:25
21:25 - 21:30	21:25 - 21:30
21:30 - 21:35	21:30 - 21:35
21:35 - 21:40	21:35 - 21:40
21:40 - 21:45	21:40 - 21:45
21:45 - 21:50	21:45 - 21:50
21:50 - 21:55	21:50 - 21:55
21:55 - 22:00	21:55 - 22:00
22:00 - 22:05	22:00 - 22:05
22:05 - 22:10	22:05 - 22:10
22:10 - 22:15	22:10 - 22:15
22:15 - 22:20	22:15 - 22:20
22:20 - 22:25	22:20 - 22:25
22:25 - 22:30	22:25 - 22:30
22:30 - 22:35	22:30 - 22:35
22:35 - 22:40	22:35 - 22:40
22:40 - 22:45	22:40 - 22:45
22:45 - 22:50	22:45 - 22:50
22:50 - 22:55	22:50 - 22:55
22:55 - 23:00	22:55 - 23:00
23:00 - 23:05	23:00 - 23:05
23:05 - 23:10	23:05 - 23:10
23:10 - 23:15	23:10 - 23:15
23:15 - 23:20	23:15 - 23:20
23:20 - 23:25	23:20 - 23:25
23:25 - 23:30	23:25 - 23:30
23:30 - 23:35	23:30 - 23:35
23:35 - 23:40	23:35 - 23:40
23:40 - 23:45	23:40 - 23:45
23:45 - 23:50	23:45 - 23:50
23:50 - 23:55	23:50 - 23:55
23:55 - 00:00	23:55 - 00:00



file times



Saved: 4/24/2026 1:43:10 AM

≡ Task #3 (0.17 pts.) - Reflection

Progress: 100%

⇒ Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

For this milestone, I learned a lot more about how to take API data and turn it into something my own app can actually use. I also got more practice with CRUD pages, like making separate pages to create, list, view, edit, and delete records. Another thing I learned was how important validation is, because even if something looks fine on the form, the server still has to check it too. I also got better with using ids in the URL, redirects, and handling stuff like duplicates or missing records.



Saved: 4/24/2026 2:09:07 AM

⇒ Task #2 (0.33 pts.) - What was the easiest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part for me was probably the basic page structure once I figured out the pattern. After I got one page working, it was easier to build the others because a lot of the logic was similar. The styling part was also not too bad since I used bootstrap, so I didn't have to do a lot of custom CSS. Once I had the table set up, some of the simpler database queries were pretty straightforward too.



Saved: 4/24/2026 2:09:27 AM

⇒ Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:

Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part was definitely dealing with the API data and making it fit my database structure. The API didn't always give back exactly what I expected, so I had to do some cleanup and also figure out how to handle duplicates. Another hard part was making sure all the validation layers worked the way they were supposed to and showing the right examples for the worksheet. There were also just a lot of little requirements in this milestone, so keeping track of everything and making sure each page matched the rubric was probably the most annoying part.



Saved: 4/24/2026 2:09:41 AM